附录 B
严格模式

ECMAScript 5 首次引入**严格模式**的概念。严格模式用于选择以更严格的条件检查 JavaScript 代码错误，可以应用到全局，也可以应用到函数内部。严格模式的好处是可以提早发现错误，因此可以捕获某些 ECMAScript 问题导致的编程错误。

理解严格模式的规则非常重要，因为未来的 ECMAScript 会逐步强制全局使用严格模式。严格模式已得到所有主流浏览器支持。

B.1 选择使用

要选择使用严格模式，需要使用严格模式**编译指示**（pragma），即一个不赋值给任何变量的字符串：

```
"use strict";
```

这样一个即使在 ECMAScript 3 中也有效的字符串，可以兼容不支持严格模式的 JavaScript 引擎。支持严格模式的引擎会启用严格模式，而不支持的引擎则会将这个编译指示当成一个未赋值的字符串字面量。

如果把这个编译指示应用到全局作用域，即函数外部，则整个脚本都会按照严格模式来解析。这意味着在最终会与其他脚本拼接为一个文件的脚本中添加了编译指示，会将该文件中的所有 JavaScript 置于严格模式之下。

也可以像下面这样只在一个函数内部开启严格模式：

```
function doSomething() {
  "use strict";
  // 其他代码
}
```

如果你不能控制页面中的所有脚本，那么建议只在经过测试的特定函数中启用严格模式。

B.2 变量

严格模式下如何创建变量及何时会创建变量都会发生变化。第一个变化是不允许意外创建全局变量。在非严格模式下，以下代码可以创建全局变量：

```
// 变量未声明
// 非严格模式：创建全局变量
// 严格模式：抛出 ReferenceError
message = "Hello world!";
```

虽然这里的 `message` 没有前置 `let` 关键字，也没有明确定义为全局对象的属性，但仍然会自动创建为全局变量。在严格模式下，给未声明的变量赋值会在执行代码时抛出 `ReferenceError`。

相关的另一个变化是无法在变量上调用 `delete`。在非严格模式下允许这样，但可能会静默失败（返

回 false)。在严格模式下，尝试删除变量会导致错误：

```
// 删除变量
// 非严格模式：静默失败
// 严格模式：抛出 ReferenceError
let color = "red";
delete color;
```

严格模式也对变量名增加了限制。具体来说，不允许变量名为 implements、interface、let、package、private、protected、public、static 和 yield。这些是目前的保留字，可能在将来的 ECMAScript 版本中用到。如果在严格模式下使用这些名称作为变量名，则会导致语法错误。

B.3　对象

在严格模式下操作对象比在非严格模式下更容易抛出错误。严格模式倾向于在非严格模式下会静默失败的情况下抛出错误，增加了开发中提前发现错误的可能性。

首先，以下几种情况下试图操纵对象属性会引发错误。

- 给只读属性赋值会抛出 TypeError。
- 在不可配置属性上使用 delete 会抛出 TypeError。
- 给不存在的对象添加属性会抛出 TypeError。

另外，与对象相关的限制也涉及通过对象字面量声明它们。在使用对象字面量时，属性名必须唯一。例如：

```
// 两个属性重名
// 非严格模式：没有错误，第二个属性生效
// 严格模式：抛出 SyntaxError
let person = {
  name: "Nicholas",
  name: "Greg"
};
```

这里的对象字面量 person 有两个叫作 name 的属性。第二个属性在非严格模式下是最终的属性。但在严格模式下，这样写是语法错误。

> **注意**　ECMAScript 6 删除了对重名属性的这个限制，即在严格模式下重复的对象字面量属性键不会抛出错误。

B.4　函数

首先，严格模式要求命名函数参数必须唯一。看下面的例子：

```
// 命名参数重名
// 非严格模式：没有错误，只有第二个参数有效
// 严格模式：抛出 SyntaxError
function sum (num, num){
  // 函数代码
}
```

在非严格模式下，这个函数声明不会抛出错误。这样可以通过名称访问第二个 num，但只能通过 arguments 访问第一个参数。

arguments 对象在严格模式下也有一些变化。在非严格模式下，修改命名参数也会修改 arguments 对象中的值。而在严格模式下，命名参数和 arguments 是相互独立的。例如：

```
// 修改命名参数的值
// 非严格模式：arguments 会反映变化
// 严格模式：arguments 不会反映变化
function showValue(value){
  value = "Foo";
  alert(value);          // "Foo"
  alert(arguments[0]);   // 非严格模式："Foo"
                         // 严格模式："Hi"
}
showValue("Hi");
```

在这个例子中，函数 showValue()有一个命名参数 value。调用这个函数时给它传入参数"Hi"，该值会赋给 value。在函数内部，value 被修改为"Foo"。在非严格模式下，这样也会修改 arguments[0]的值，但在严格模式下则不会。

另一个变化是去掉了 arguments.callee 和 arguments.caller。在非严格模式下，它们分别引用函数本身和调用函数。在严格模式下，访问这两个属性中的任何一个都会抛出 TypeError。例如：

```
// 访问 arguments.callee
// 非严格模式：没问题
// 严格模式：抛出 TypeError
function factorial(num){
  if (num <= 1) {
    return 1;
  } else {
    return num * arguments.callee(num-1)
  }
}
let result = factorial(5);
```

类似地，读或写函数的 caller 或 callee 属性也会抛出 TypeError。因此对这个例子而言，访问 factorial.caller 和 factorial.callee 也会抛出错误。

另外，与变量一样，严格模式也限制了函数的命名，不允许函数名为 implements、interface、let、package、private、protected、public、static 和 yield。

关于函数的最后一个变化是不允许函数声明，除非它们位于脚本或函数的顶级。这意味着在 if 语句中声明的函数现在是个语法错误：

```
// 在 if 语句中声明函数
// 非严格模式：函数提升至 if 语句外部
// 严格模式：抛出 SyntaxError
if (true){
  function doSomething(){
    // ...
  }
}
```

所有浏览器在非严格模式下都支持这个语法，但在严格模式下则会抛出语法错误。

B.4.1　函数参数

ES6 增加了剩余操作符、解构操作符和默认参数，为函数组织、结构和定义参数提供了强大的支持。ECMAScript 7 增加了一条限制，要求使用任何上述先进参数特性的函数内部都不能使用严格模式，否则

会抛出错误。不过，全局严格模式还是允许的。

```
// 可以
function foo(a, b, c) {
  "use strict";
}

// 不可以
function bar(a, b, c='d') {
  "use strict";
}

// 不可以
function baz({a, b, c}) {
  "use strict";
}

// 不可以
function qux(a, b, ...c) {
  "use strict";
}
```

ES6 增加的这些新特性期待参数与函数体在相同模式下进行解析。如果允许编译指示"use strict"出现在函数体内，JavaScript 解析器就需要在解析函数参数之前先检查函数体内是否存在这个编译指示，而这会带来很多问题。为此，ES7 规范增加了这个约定，目的是让解析器在解析函数之前就确切知道该使用什么模式。

B.4.2　eval()

eval()函数在严格模式下也有变化。最大的变化是 eval()不会再在包含上下文中创建变量或函数。例如：

```
// 使用 eval()创建变量
// 非严格模式：警告框显示 10
// 严格模式：调用 alert(x)时抛出 ReferenceError
function doSomething(){
  eval("let x = 10");
  alert(x);
}
```

以上代码在非严格模式下运行时，会在 doSomething()函数内部创建局部变量 x，然后 alert()会显示这个变量的值。在严格模式下，调用 eval()不会在 doSomething()中创建变量 x，由于 x 没有声明，alert()会抛出 ReferenceError。

变量和函数可以在 eval()中声明，但它们会位于代码执行期间的一个特殊的作用域里，代码执行完毕就会销毁。因此，以下代码就不会出错：

```
"use strict";
let result = eval("let x = 10, y = 11; x + y");
alert(result);    // 21
```

这里在 eval()中声明了变量 x 和 y，将它们相加后返回得到的结果。变量 result 会包含 x 和 y 相加的结果 21，虽然 x 和 y 在调用 alert()时已经不存在了，但不影响结果的显示。

B.4.3　eval 与 arguments

严格模式明确不允许使用 eval 和 arguments 作为标识符和操作它们的值。例如：

```
// 将 eval 和 arguments 重新定义为变量
// 非严格模式：可以，没有错误
// 严格模式：抛出 SyntaxError
let eval = 10;
let arguments = "Hello world!";
```

在非严格模式下，可以重写 eval 和 arguments。在严格模式下，这样会导致语法错误。不能用它们作为标识符，这意味着下面这些情况都会抛出语法错误：

- 使用 let 声明；
- 赋予其他值；
- 修改其包含的值，如使用++；
- 用作函数名；
- 用作函数参数名；
- 在 try/catch 语句中用作异常名称。

B.5　this 强制转型

JavaScript 中最大的一个安全问题，也是最令人困惑的一个问题，就是在某些情况下 this 的值是如何确定的。使用函数的 apply()或 call()方法时，在非严格模式下 null 或 undefined 值会被强制转型为全局对象。在严格模式下，则始终以指定值作为函数 this 的值，无论指定的是什么值。例如：

```
// 访问属性
// 非严格模式：访问全局属性
// 严格模式：抛出错误，因为 this 值为 null
let color = "red";
function displayColor() {
  alert(this.color);
}
displayColor.call(null);
```

这里在调用 displayColor.call()时传入 null 作为 this 的值，在非严格模式下该函数的 this 值是全局对象。结果会显示"red"。在严格模式下，该函数的 this 值是 null，因此在访问 null 的属性时会抛出错误。

通常，函数会将其 this 的值转型为一种对象类型，这种行为经常被称为“装箱”（boxing）。这意味着原始值会转型为它们的包装对象类型。

```
function foo() {
  console.log(this);
}

foo.call();  // Window {}
foo.call(2); // Number {2}
```

在严格模式下执行以上代码时，this 的值不会再“装箱”：

```
function foo() {
  "use strict";
  console.log(this);
```

```
}

foo.call();  // undefined
foo.call(2); // 2
```

B.6　类与模块

类和模块都是 ECMAScript 6 新增的代码容器特性。在之前的 ECMAScript 版本中没有类和模块这两个概念，因此不用考虑从语法上兼容之前的 ECMAScript 版本。为此，TC39 委员会决定在 ES6 类和模块中定义的所有代码默认都处于严格模式。

对于类，这包括类声明和类表达式，构造函数、实例方法、静态方法、获取方法和设置方法都在严格模式下。对于模块，所有在其内部定义的代码都处于严格模式。

B.7　其他变化

严格模式下还有其他一些需要注意的变化。首先是消除 `with` 语句。`with` 语句改变了标识符解析时的方式，严格模式下为简单起见已去掉了这个语法。在严格模式下使用 `with` 会导致语法错误：

```
// 使用 with 语句
// 非严格模式：允许
// 严格模式：抛出 SyntaxError
with(location) {
  alert(href);
}
```

严格模式也从 JavaScript 中去掉了**八进制字面量**。八进制字面量以前导 0 开始，一直以来是很多错误的源头。在严格模式下使用八进制字面量被认为是无效语法：

```
// 使用八进制字面量
// 非严格模式：值为 8
// 严格模式：抛出 SyntaxError
let value = 010;
```

ECMAScript 5 修改了非严格模式下的 `parseInt()`，将八进制字面量当作带前导 0 的十进制字面量。例如：

```
// 在 parseInt() 中使用八进制字面量
// 非严格模式：值为 8
// 严格模式：值为 10
let value = parseInt("010");
```